Test Applications with FastAPI and SQLModel

To finish this group of chapters about **FastAPI** with **SQLModel**, let's now learn how to implement automated tests for an application using FastAPI with SQLModel. ✅

Including the tips and tricks. 🎁

FastAPI Application

Let's work with one of the **simpler** FastAPI applications we built in the previous chapters.

All the same **concepts**, **tips** and **tricks** will apply to more complex applications as well.

We will use the application with the hero models, but without team models, and we will use the dependency to get a **session**.

Now we will see how useful it is to have this session dependency. ✨

👀 **Full file preview**

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes
```

```python
@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

File Structure

Now we will have a Python project with multiple files, one file `main.py` with all the application, and one file `test_main.py` with the tests, with the same ideas from Code Structure and Multiple Files.

The file structure is:

```
.
├── project
    ├── __init__.py
    ├── main.py
    └── test_main.py
```

Testing FastAPI Applications

If you haven't done testing in FastAPI applications, first check the FastAPI docs about Testing.

Then, we can continue here, the first step is to install the dependencies, `requests` and `pytest`.

Make sure you create a virtual environment, activate it, and then install them, for example with:

```bash
```

Basic Tests Code

Let's start with a simple test, with just the basic test code we need the check that the **FastAPI** application is creating a new hero correctly.

```
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine

from .main import app, get_session  ❶


def test_create_hero():
        # Some code here omitted, we will see it later 👈
        client = TestClient(app)  ❷

        response = client.post(  ❸
            "/heroes/", json={"name": "Deadpond", "secret_name": "Dive Wilson"}
        )
        # Some code here omitted, we will see it later 👈
        data = response.json()  ❹

        assert response.status_code == 200  ❺
        assert data["name"] == "Deadpond"  ❻
        assert data["secret_name"] == "Dive Wilson"  ❼
        assert data["age"] is None  ❽
        assert data["id"] is not None  ❾

# Code below omitted 👇
```

❶ Import the `app` from the the `main` module.

❷ We create a `TestClient` for the FastAPI `app` and put it in the variable `client`.

❸ Then we use use this `client` to **talk to the API** and send a `POST` HTTP operation, creating a new hero.

❹ Then we get the **JSON data** from the response and put it in the variable `data`.

❺ Next we start testing the results with `assert` statements, we check that the status code of the response is `200`.

❻ We check that the `name` of the hero created is `"Deadpond"`.

❼ We check that the `secret_name` of the hero created is `"Dive Wilson"`.

❽ We check that the `age` of the hero created is `None`, because we didn't send an age.

❾ We check that the hero created has an `id` created by the database, so it's not `None`.

> **Tip**
>
> Check out the number bubbles to see what is done by each line of code.

That's the **core** of the code we need for all the tests later.

But now, we need to deal with a bit of logistics and details we are not paying attention to just yet. 🤓

Testing Database

This test looks fine, but there's a problem.

If we run it, it will use the same **production database** that we are using to store our very important **heroes**, and we will end up adding unnecessary data to it, or even worse, in future tests we could end up removing production data.

So, we should use an independent **testing database**, just for the tests.

To do this, we need to change the URL used for the database.

But when the code for the API is executed, it gets a **session** that is already connected to an **engine**, and the **engine** is already using a specific database URL.

Even if we import the variable from the `main` module and change its value just for the tests, by that point the **engine** is already created with the original value.

But all our API *path operations* get the *session* using a FastAPI **dependency**, and we can override dependencies in tests.

Here's where dependencies start to help a lot.

Override a Dependency

Let's override the `get_session()` dependency for the tests.

This dependency is used by all the *path operations* to get the **SQLModel** session object.

We will override it to use a different **session** object just for the tests.

That way we protect the production database and we have better control of the data we are testing.

```
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine

from .main import app, get_session  ❶


def test_create_hero():
        # Some code here omitted, we will see it later 👈
        def get_session_override():  ❷
            return session  ❸

        app.dependency_overrides[get_session] = get_session_override  ❹

        client = TestClient(app)

        response = client.post(
            "/heroes/", json={"name": "Deadpond", "secret_name": "Dive Wilson"}
        )
        app.dependency_overrides.clear()  ❺
        data = response.json()

        assert response.status_code == 200
        assert data["name"] == "Deadpond"
        assert data["secret_name"] == "Dive Wilson"
        assert data["age"] is None
        assert data["id"] is not None

# Code below omitted 👇
```

❶ Import the `get_session` dependency from the the `main` module.

❷ Define the new function that will be the new **dependency override**.

❸ This function will return a different **session** than the one that would be returned by the original `get_session` function.

We haven't seen how this new **session** object is created yet, but the point is that this is a different session than the original one from the app.

This session is attached to a different **engine**, and that different **engine** uses a different URL, for a database just for testing.

We haven't defined that new **URL** nor the new **engine** yet, but here we already see the that this object `session` will override the one returned by the original dependency `get_session()`.

4. Then, the FastAPI `app` object has an attribute `app.dependency_overrides`.

   This attribute is a dictionary, and we can put dependency overrides in it by passing, as the **key**, the **original dependency function**, and as the **value**, the **new overriding dependency function**.

   So, here we are telling the FastAPI app to use `get_session_override` instead of `get_session` in all the places in the code that depend on `get_session`, that is, all the parameters with something like:

```
session: Session = Depends(get_session)
```

5. After we are done with the dependency override, we can restore the application back to normal, by removing all the values in this dictionary `app.dependency_overrides`.

   This way whenever a *path operation function* needs the dependency FastAPI will use the original one instead of the override.

> **Tip**
>
> Check out the number bubbles to see what is done by each line of code.

Create the Engine and Session for Testing

Now let's create that **session** object that will be used during testing.

It will use its own **engine**, and this new engine will use a new URL for the testing database:

```
sqlite:///testing.db
```

So, the testing database will be in the file `testing.db`.

```
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine

from .main import app, get_session  1


def test_create_hero():
    engine = create_engine(  2
        "sqlite:///testing.db", connect_args={"check_same_thread": False}
    )
    SQLModel.metadata.create_all(engine)  3

    with Session(engine) as session:  4

        def get_session_override():
            return session  5

        app.dependency_overrides[get_session] = get_session_override  4

        client = TestClient(app)

        response = client.post(
            "/heroes/", json={"name": "Deadpond", "secret_name": "Dive Wilson"}
        )
        app.dependency_overrides.clear()
        data = response.json()
```

```
        assert response.status_code == 200
        assert data["name"] == "Deadpond"
        assert data["secret_name"] == "Dive Wilson"
        assert data["age"] is None
        assert data["id"] is not None
    6
```

1. Here's a subtle thing to notice.

   Remember that Order Matters and we need to make sure all the **SQLModel** models are already defined and **imported** before calling `.create_all()`.

   IN this line, by importing something, *anything*, from `.main`, the code in `.main` will be executed, including the definition of the **table models**, and that will automatically register them in `SQLModel.metadata`.

2. Here we create a new **engine**, completely different from the one in `main.py`.

   This is the engine we will use for the tests.

   We use the new URL of the database for tests:

   ```
   sqlite:///testing.db
   ```

   And again, we use the connection argument `check_same_thread=False`.

3. Then we call:

   ```
   SQLModel.metadata.create_all(engine)
   ```

   ...to make sure we create all the tables in the new testing database.

   The **table models** are registered in `SQLModel.metadata` just because we imported *something* from `.main`, and the code in `.main` was executed, creating the classes for the **table models** and automatically registering them in `SQLModel.metadata`.

   So, by the point we call this method, the **table models** are already registered there. 💯

4. Here's where we create the custom **session** object for this test in a `with` block.

   It uses the new custom **engine** we created, so anything that uses this session will be using the testing database.

5. Now, back to the dependency override, it is just returning the same **session** object from outside, that's it, that's the whole trick.

6. By this point, the testing **session** `with` block finishes, and the session is closed, the file is closed, etc.

Import Table Models

Here we create all the tables in the testing database with:

```
SQLModel.metadata.create_all(engine)
```

But remember that Order Matters and we need to make sure all the **SQLModel** models are already defined and **imported** before calling `.create_all()`.

In this case, it all works for a little subtlety that deserves some attention.

Because we import something, *anything*, from `.main`, the code in `.main` will be executed, including the definition of the **table models**, and that will automatically register them in `SQLModel.metadata`.

That way, when we call `.create_all()` all the **table models** are correctly registered in `SQLModel.metadata` and it will all work. 👌

Memory Database

Now we are not using the production database. Instead, we use a **new testing database** with the `testing.db` file, which is great.

But SQLite also supports having an **in memory** database. This means that all the database is only in memory, and it is never saved in a file on disk.

After the program terminates, **the in-memory database is deleted**, so it wouldn't help much for a production database.

But **it works great for testing**, because it can be quickly created before each test, and quickly removed after each test. ✅

And also, because it never has to write anything to a file and it's all just in memory, it will be even faster than normally. 🏎️

Other alternatives and ideas 👀

Before arriving at the idea of using an **in-memory database** we could have explored other alternatives and ideas.

The first is that we are not deleting the file after we finish the test, so the next test could have **leftover data**. So, the right thing would be to delete the file right after finishing the test. 🔥

But if each test has to create a new file and then delete it afterwards, running all the tests could be **a bit slow**.

Right now, we have a file `testing.db` that is used by all the tests (we only have one test now, but we will have more).

So, if we tried to run the tests at the same time **in parallel** to try to speed things up a bit, they would clash trying to use the *same* `testing.db` file.

Of course, we could also fix that, using some **random name** for each testing database file... but in the case of SQLite, we have an even better alternative by just using an **in-memory database**. ✨

Configure the In-Memory Database

Let's update our code to use the in-memory database.

We just have to change a couple of parameters in the **engine**.

```python
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine
from sqlmodel.pool import StaticPool  ❶

from .main import app, get_session


def test_create_hero():
    engine = create_engine(
        "sqlite://",  ❷
        connect_args={"check_same_thread": False},
        poolclass=StaticPool,  ❸
    )

# Code below omitted 👇
```

❶ Import `StaticPool` from `sqlmodel`, we will use it in a bit.

❷ For the **SQLite URL**, don't write any file name, leave it empty.

So, instead of:

```
sqlite:///testing.db
```

...just write:

```
sqlite://
```

This is enough to tell **SQLModel** (actually SQLAlchemy) that we want to use an **in-memory SQLite database**.

3 Remember that we told the **low-level** library in charge of communicating with SQLite that we want to be able to **access the database from different threads** with `check_same_thread=False`?

Now that we use an **in-memory database**, we need to also tell SQLAlchemy that we want to be able to use the **same in-memory database** object from different threads.

We tell it that with the `poolclass=StaticPool` parameter.

> **Info**
>
> You can read more details in the SQLAlchemy documentation about Using a Memory Database in Multiple Threads

> **Tip**
>
> Check out the number bubbles to see what is done by each line of code.

That's it, now the test will run using the **in-memory database**, which will be faster and probably safer.

And all the other tests can do the same.

Boilerplate Code

Great, that works, and you could replicate all that process in each of the test functions.

But we had to add a lot of **boilerplate code** to handle the custom database, creating it in memory, the custom session, and the dependency override.

Do we really have to duplicate all that for **each test**? No, we can do better! 😎

We are using **pytest** to run the tests. And pytest also has a very similar concept to the **dependencies in FastAPI**.

> **Info**
>
> In fact, pytest was one of the things that inspired the design of the dependencies in FastAPI.

It's a way for us to declare some **code that should be run before** each test and **provide a value** for the test function (that's pretty much the same as FastAPI dependencies).

In fact, it also has the same trick of allowing to use `yield` instead of `return` to provide the value, and then **pytest** makes sure that the code after `yield` is executed *after* the function with the test is done.

In pytest, these things are called **fixtures** instead of *dependencies*.

Let's use these **fixtures** to improve our code and reduce de duplicated boilerplate for the next tests.

Pytest Fixtures

You can read more about them in the pytest docs for fixtures, but I'll give you a short example for what we need here.

Let's see the first code example with a fixture:

```
import pytest  ❶
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine
from sqlmodel.pool import StaticPool

from .main import app, get_session


@pytest.fixture(name="session")  ❷
def session_fixture():  ❸
    engine = create_engine(
        "sqlite://", connect_args={"check_same_thread": False}, poolclass=StaticPool
    )
    SQLModel.metadata.create_all(engine)
    with Session(engine) as session:
        yield session  ❹


def test_create_hero(session: Session):  ❺
    def get_session_override():
        return session  ❻

    app.dependency_overrides[get_session] = get_session_override

    client = TestClient(app)

    response = client.post(
        "/heroes/", json={"name": "Deadpond", "secret_name": "Dive Wilson"}
    )
    app.dependency_overrides.clear()
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == "Deadpond"
    assert data["secret_name"] == "Dive Wilson"
    assert data["age"] is None
    assert data["id"] is not None
```

❶ Import `pytest`.

❷ Use the `@pytest.fixture()` decorator on top of the function to tell pytest that this is a **fixture** function (equivalent to a FastAPI dependency).

We also give it a name of `"session"`, this will be important in the testing function.

❸ Create the fixture function. This is equivalent to a FastAPI dependency function.

In this fixture we create the custom **engine**, with the in-memory database, we create the tables, and we create the **session**.

Then we `yield` the `session` object.

❹ The thing that we `return` or `yield` is what will be available to the test function, in this case, the `session` object.

Here we use `yield` so that **pytest** comes back to execute "the rest of the code" in this function once the testing function is done.

We don't have any more visible "rest of the code" after the `yield`, but we have the end of the `with` block that will close the **session**.

By using `yield`, pytest will:

- ❺ run the first part
- ❻ create the **session** object
- ❼ give it to the test function

8. run the test function
9. once the test function is done, it will continue here, right after the `yield`, and will correctly close the **session** object in the end of the `with` block.

10. Now, in the test function, to tell **pytest** that this test wants to get the fixture, instead of declaring something like in FastAPI with:

```
session: Session = Depends(session_fixture)
```

...the way we tell pytest what is the fixture that we want is by using the **exact same name** of the fixture.

In this case, we named it `session`, so the parameter has to be exactly named `session` for it to work.

We also add the type annotation `session: Session` so that we can get autocompletion and inline error checks in our editor.

11. Now in the dependency override function, we just return the same `session` object that came from outside it.

The `session` object comes from the parameter passed to the test function, and we just re-use it and return it here in the dependency override.

> **Tip**
>
> Check out the number bubbles to see what is done by each line of code.

pytest fixtures work in a very similar way to FastAPI dependencies, but have some minor differences:

- In pytest fixtures, we need to add a decorator of `@pytest.fixture()` on top.
- To use a pytest fixture in a function, we have to declare the parameter with the **exact same name**. In FastAPI we have to **explicitly use `Depends()`** with the actual function inside it.

But apart from the way we declare them and how we tell the framework that we want to have them in the function, they **work in a very similar way**.

Now we create lot's of tests and re-use that same fixture in all of them, saving us that **boilerplate code**.

pytest will make sure to run them right before (and finish them right after) each test function. So, each test function will actually have its own database, engine, and session.

Client Fixture

Awesome, that fixture helps us prevent a lot of duplicated code.

But currently, we still have to write some code in the test function that will be repetitive for other tests, right now we:

- create the **dependency override**
- put it in the `app.dependency_overrides`
- create the `TestClient`
- Clear the dependency override(s) after making the request

That's still gonna be repetitive in the other future tests. Can we improve it? Yes! 🎉

Each **pytest** fixture (the same way as **FastAPI** dependencies), can require other fixtures.

So, we can create a **client fixture** that will be used in all the tests, and it will itself require the **session fixture**.

```
import pytest
from fastapi.testclient import TestClient
```

```python
from sqlmodel import Session, SQLModel, create_engine
from sqlmodel.pool import StaticPool

from .main import app, get_session


@pytest.fixture(name="session")
def session_fixture():
    engine = create_engine(
        "sqlite://", connect_args={"check_same_thread": False}, poolclass=StaticPool
    )
    SQLModel.metadata.create_all(engine)
    with Session(engine) as session:
        yield session


@pytest.fixture(name="client")  1
def client_fixture(session: Session):  2
    def get_session_override():  3
        return session

    app.dependency_overrides[get_session] = get_session_override  4

    client = TestClient(app)  5
    yield client  6
    app.dependency_overrides.clear()  7


def test_create_hero(client: TestClient):  8
    response = client.post(
        "/heroes/", json={"name": "Deadpond", "secret_name": "Dive Wilson"}
    )
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == "Deadpond"
    assert data["secret_name"] == "Dive Wilson"
    assert data["age"] is None
    assert data["id"] is not None
```

1. Create the new fixture named `"client"`.
2. This **client fixture**, in turn, also requires the **session fixture**.
3. Now we create the **dependency override** inside the client fixture.
4. Set the **dependency override** in the `app.dependency_overrides` dictionary.
5. Create the `TestClient` with the **FastAPI** `app`.
6. `yield` the `TestClient` instance.

   By using `yield`, after the test function is done, pytest will come back to execute the rest of the code after `yield`.
7. This is the cleanup code, after `yield`, and after the test function is done.

   Here we clear the dependency overrides (here it's only one) in the FastAPI `app`.
8. Now the test function requires the **client fixture**.

   And inside the test function, the code is quite **simple**, we just use the `TestClient` to make requests to the API, check the data, and that's it.

   The fixtures take care of all the **setup** and **cleanup** code.

> **Tip**
>
> Check out the number bubbles to see what is done by each line of code.

Now we have a **client fixture** that, in turn, uses the **session fixture**.

And in the actual test function, we just have to declare that we require this **client fixture**.

Add More Tests

At this point, it all might seem like we just did a lot of changes for nothing, to get **the same result**. 🤔

But normally we will create **lots of other test functions**. And now all the boilerplate and complexity is **written only once**, in those two fixtures.

Let's add some more tests:

Python 3.10+

```python
# Code above omitted 👆

def test_create_hero(client: TestClient):
    response = client.post(
        "/heroes/", json={"name": "Deadpond", "secret_name": "Dive Wilson"}
    )
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == "Deadpond"
    assert data["secret_name"] == "Dive Wilson"
    assert data["age"] is None
    assert data["id"] is not None


def test_create_hero_incomplete(client: TestClient):
    # No secret_name
    response = client.post("/heroes/", json={"name": "Deadpond"})
    assert response.status_code == 422


def test_create_hero_invalid(client: TestClient):
    # secret_name has an invalid type
    response = client.post(
        "/heroes/",
        json={
            "name": "Deadpond",
            "secret_name": {"message": "Do you wanna know my secret identity?"},
        },
    )
    assert response.status_code == 422

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
import pytest
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine
from sqlmodel.pool import StaticPool

from .main import Hero, app, get_session


@pytest.fixture(name="session")
def session_fixture():
    engine = create_engine(
        "sqlite://", connect_args={"check_same_thread": False}, poolclass=StaticPool
    )
    SQLModel.metadata.create_all(engine)
    with Session(engine) as session:
        yield session


@pytest.fixture(name="client")
def client_fixture(session: Session):
    def get_session_override():
        return session

    app.dependency_overrides[get_session] = get_session_override
    client = TestClient(app)
    yield client
    app.dependency_overrides.clear()


def test_create_hero(client: TestClient):
    response = client.post(
        "/heroes/", json={"name": "Deadpond", "secret_name": "Dive Wilson"}
    )
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == "Deadpond"
    assert data["secret_name"] == "Dive Wilson"
    assert data["age"] is None
    assert data["id"] is not None


def test_create_hero_incomplete(client: TestClient):
    # No secret_name
    response = client.post("/heroes/", json={"name": "Deadpond"})
    assert response.status_code == 422


def test_create_hero_invalid(client: TestClient):
    # secret_name has an invalid type
    response = client.post(
        "/heroes/",
        json={
            "name": "Deadpond",
            "secret_name": {"message": "Do you wanna know my secret identity?"},
        },
    )
    assert response.status_code == 422


def test_read_heroes(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    session.add(hero_1)
    session.add(hero_2)
    session.commit()

    response = client.get("/heroes/")
    data = response.json()
```

```python
    assert response.status_code == 200

    assert len(data) == 2
    assert data[0]["name"] == hero_1.name
    assert data[0]["secret_name"] == hero_1.secret_name
    assert data[0]["age"] == hero_1.age
    assert data[0]["id"] == hero_1.id
    assert data[1]["name"] == hero_2.name
    assert data[1]["secret_name"] == hero_2.secret_name
    assert data[1]["age"] == hero_2.age
    assert data[1]["id"] == hero_2.id


def test_read_hero(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    session.add(hero_1)
    session.commit()

    response = client.get(f"/heroes/{hero_1.id}")
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == hero_1.name
    assert data["secret_name"] == hero_1.secret_name
    assert data["age"] == hero_1.age
    assert data["id"] == hero_1.id


def test_update_hero(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    session.add(hero_1)
    session.commit()

    response = client.patch(f"/heroes/{hero_1.id}", json={"name": "Deadpuddle"})
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == "Deadpuddle"
    assert data["secret_name"] == "Dive Wilson"
    assert data["age"] is None
    assert data["id"] == hero_1.id


def test_delete_hero(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    session.add(hero_1)
    session.commit()

    response = client.delete(f"/heroes/{hero_1.id}")

    hero_in_db = session.get(Hero, hero_1.id)

    assert response.status_code == 200

    assert hero_in_db is None
```

Tip

It's always **good idea** to not only test the normal case, but also that **invalid data**, **errors**, and **corner cases** are handled correctly.

That's why we add these two extra tests here.

Now, any additional test functions can be as **simple** as the first one, they just have to **declare the `client` parameter** to get the `TestClient` **fixture** with all the database stuff setup. Nice! 😎

Why Two Fixtures

Now, seeing the code, we could think, why do we put **two fixtures** instead of **just one** with all the code? And that makes total sense!

For these examples, **that would have been simpler**, there's no need to separate that code into two fixtures for them...

But for the next test function, we will require **both fixtures**, the **client** and the **session**.

Python 3.10+

```python
import pytest
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine
from sqlmodel.pool import StaticPool

from .main import Hero, app, get_session

# Code here omitted 👈

def test_read_heroes(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    session.add(hero_1)
    session.add(hero_2)
    session.commit()

    response = client.get("/heroes/")
    data = response.json()

    assert response.status_code == 200

    assert len(data) == 2
    assert data[0]["name"] == hero_1.name
    assert data[0]["secret_name"] == hero_1.secret_name
    assert data[0]["age"] == hero_1.age
    assert data[0]["id"] == hero_1.id
    assert data[1]["name"] == hero_2.name
    assert data[1]["secret_name"] == hero_2.secret_name
    assert data[1]["age"] == hero_2.age
    assert data[1]["id"] == hero_2.id

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
import pytest
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine
from sqlmodel.pool import StaticPool

from .main import Hero, app, get_session


@pytest.fixture(name="session")
def session_fixture():
    engine = create_engine(
        "sqlite://", connect_args={"check_same_thread": False}, poolclass=StaticPool
    )
    SQLModel.metadata.create_all(engine)
    with Session(engine) as session:
        yield session


@pytest.fixture(name="client")
def client_fixture(session: Session):
    def get_session_override():
        return session

    app.dependency_overrides[get_session] = get_session_override
    client = TestClient(app)
    yield client
    app.dependency_overrides.clear()


def test_create_hero(client: TestClient):
    response = client.post(
        "/heroes/", json={"name": "Deadpond", "secret_name": "Dive Wilson"}
    )
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == "Deadpond"
    assert data["secret_name"] == "Dive Wilson"
    assert data["age"] is None
    assert data["id"] is not None


def test_create_hero_incomplete(client: TestClient):
    # No secret_name
    response = client.post("/heroes/", json={"name": "Deadpond"})
    assert response.status_code == 422


def test_create_hero_invalid(client: TestClient):
    # secret_name has an invalid type
    response = client.post(
        "/heroes/",
        json={
            "name": "Deadpond",
            "secret_name": {"message": "Do you wanna know my secret identity?"},
        },
    )
    assert response.status_code == 422


def test_read_heroes(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    session.add(hero_1)
    session.add(hero_2)
    session.commit()

    response = client.get("/heroes/")
    data = response.json()
```

```python
    assert response.status_code == 200

    assert len(data) == 2
    assert data[0]["name"] == hero_1.name
    assert data[0]["secret_name"] == hero_1.secret_name
    assert data[0]["age"] == hero_1.age
    assert data[0]["id"] == hero_1.id
    assert data[1]["name"] == hero_2.name
    assert data[1]["secret_name"] == hero_2.secret_name
    assert data[1]["age"] == hero_2.age
    assert data[1]["id"] == hero_2.id


def test_read_hero(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    session.add(hero_1)
    session.commit()

    response = client.get(f"/heroes/{hero_1.id}")
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == hero_1.name
    assert data["secret_name"] == hero_1.secret_name
    assert data["age"] == hero_1.age
    assert data["id"] == hero_1.id


def test_update_hero(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    session.add(hero_1)
    session.commit()

    response = client.patch(f"/heroes/{hero_1.id}", json={"name": "Deadpuddle"})
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == "Deadpuddle"
    assert data["secret_name"] == "Dive Wilson"
    assert data["age"] is None
    assert data["id"] == hero_1.id


def test_delete_hero(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    session.add(hero_1)
    session.commit()

    response = client.delete(f"/heroes/{hero_1.id}")

    hero_in_db = session.get(Hero, hero_1.id)

    assert response.status_code == 200

    assert hero_in_db is None
```

In this test function, we want to check that the *path operation* to **read a list of heroes** actually sends us heroes.

But if the **database is empty**, we would get an **empty list**, and we wouldn't know if the hero data is being sent correctly or not.

But we can **create some heroes** in the testing database right before sending the API request. ✨

And because we are using the **testing database**, we don't affect anything by creating heroes for the test.

To do it, we have to:

- import the `Hero` model
- require both fixtures, the **client** and the **session**
- create some heroes and save them in the database using the **session**

After that, we can send the request and check that we actually got the data back correctly from the database. 💯

Here's the important detail to notice: we can require fixtures in other fixtures **and also** in the test functions.

The function for the **client fixture** and the actual testing function will **both** receive the same **session**.

Add the Rest of the Tests

Using the same ideas, requiring the fixtures, creating data that we need for the tests, etc., we can now add the rest of the tests. They look quite similar to what we have done up to now.

Python 3.10+

```python
# Code above omitted 👆

def test_read_hero(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    session.add(hero_1)
    session.commit()

    response = client.get(f"/heroes/{hero_1.id}")
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == hero_1.name
    assert data["secret_name"] == hero_1.secret_name
    assert data["age"] == hero_1.age
    assert data["id"] == hero_1.id


def test_update_hero(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    session.add(hero_1)
    session.commit()

    response = client.patch(f"/heroes/{hero_1.id}", json={"name": "Deadpuddle"})
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == "Deadpuddle"
    assert data["secret_name"] == "Dive Wilson"
    assert data["age"] is None
    assert data["id"] == hero_1.id


def test_delete_hero(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    session.add(hero_1)
    session.commit()

    response = client.delete(f"/heroes/{hero_1.id}")

    hero_in_db = session.get(Hero, hero_1.id)

    assert response.status_code == 200

    assert hero_in_db is None
```

👀 Full file preview

Python 3.10+

```python
import pytest
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine
from sqlmodel.pool import StaticPool

from .main import Hero, app, get_session


@pytest.fixture(name="session")
def session_fixture():
    engine = create_engine(
        "sqlite://", connect_args={"check_same_thread": False}, poolclass=StaticPool
    )
    SQLModel.metadata.create_all(engine)
    with Session(engine) as session:
        yield session


@pytest.fixture(name="client")
def client_fixture(session: Session):
    def get_session_override():
        return session

    app.dependency_overrides[get_session] = get_session_override
    client = TestClient(app)
    yield client
    app.dependency_overrides.clear()


def test_create_hero(client: TestClient):
    response = client.post(
        "/heroes/", json={"name": "Deadpond", "secret_name": "Dive Wilson"}
    )
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == "Deadpond"
    assert data["secret_name"] == "Dive Wilson"
    assert data["age"] is None
    assert data["id"] is not None


def test_create_hero_incomplete(client: TestClient):
    # No secret_name
    response = client.post("/heroes/", json={"name": "Deadpond"})
    assert response.status_code == 422


def test_create_hero_invalid(client: TestClient):
    # secret_name has an invalid type
    response = client.post(
        "/heroes/",
        json={
            "name": "Deadpond",
            "secret_name": {"message": "Do you wanna know my secret identity?"},
        },
    )
    assert response.status_code == 422


def test_read_heroes(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    session.add(hero_1)
    session.add(hero_2)
    session.commit()

    response = client.get("/heroes/")
    data = response.json()
```

```python
    assert response.status_code == 200

    assert len(data) == 2
    assert data[0]["name"] == hero_1.name
    assert data[0]["secret_name"] == hero_1.secret_name
    assert data[0]["age"] == hero_1.age
    assert data[0]["id"] == hero_1.id
    assert data[1]["name"] == hero_2.name
    assert data[1]["secret_name"] == hero_2.secret_name
    assert data[1]["age"] == hero_2.age
    assert data[1]["id"] == hero_2.id


def test_read_hero(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    session.add(hero_1)
    session.commit()

    response = client.get(f"/heroes/{hero_1.id}")
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == hero_1.name
    assert data["secret_name"] == hero_1.secret_name
    assert data["age"] == hero_1.age
    assert data["id"] == hero_1.id


def test_update_hero(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    session.add(hero_1)
    session.commit()

    response = client.patch(f"/heroes/{hero_1.id}", json={"name": "Deadpuddle"})
    data = response.json()

    assert response.status_code == 200
    assert data["name"] == "Deadpuddle"
    assert data["secret_name"] == "Dive Wilson"
    assert data["age"] is None
    assert data["id"] == hero_1.id


def test_delete_hero(session: Session, client: TestClient):
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    session.add(hero_1)
    session.commit()

    response = client.delete(f"/heroes/{hero_1.id}")

    hero_in_db = session.get(Hero, hero_1.id)

    assert response.status_code == 200

    assert hero_in_db is None
```

Run the Tests

Now we can run the tests with `pytest` and see the results:

```bash
```

Recap

Did you read all that? Wow, I'm impressed! 😎

Adding tests to your application will give you a lot of **certainty** that everything is **working correctly**, as you intended.

And tests will be notoriously useful when **refactoring** your code, **changing things**, **adding features**. Because tests can help catch a lot of errors that can be easily introduced by refactoring.

And they will give you the confidence to work faster and **more efficiently**, because you know that you are checking if you are **not breaking anything**. 😅

I think tests are one of those things that bring your code and you as a developer to the next professional level. 😎

And if you read and studied all this, you already know a lot of the advanced ideas and tricks that took me years to learn. 🚀